Day 105: Decision Trees From Scratch — Entropy and Gini on Real Data

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 1 | Complete study guide

Watch: Decision and Classification Trees, Clearly Explained!!! (StatQuest) | Gini & Entropy versus misclassification error (Sebastian Raschka)

Script: `day105_decision_trees.py` · every number in this guide is copied by program from `day105_run_output.txt`, the real output of that script.

A decision tree written from scratch in NumPy, on 569 real breast tumours. Its fast split search matches brute force in 30 of 30 checks. Deeper trees fit the training data perfectly (1.0000) while test accuracy stops improving at 0.9415; cross-validation picks depth 2. The tree is unstable: on 30 resamples of the same data the first question changes between five features, and a vote of those 30 trees (0.9649) beats the average single tree (0.9376), which is Day 107's random forest.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

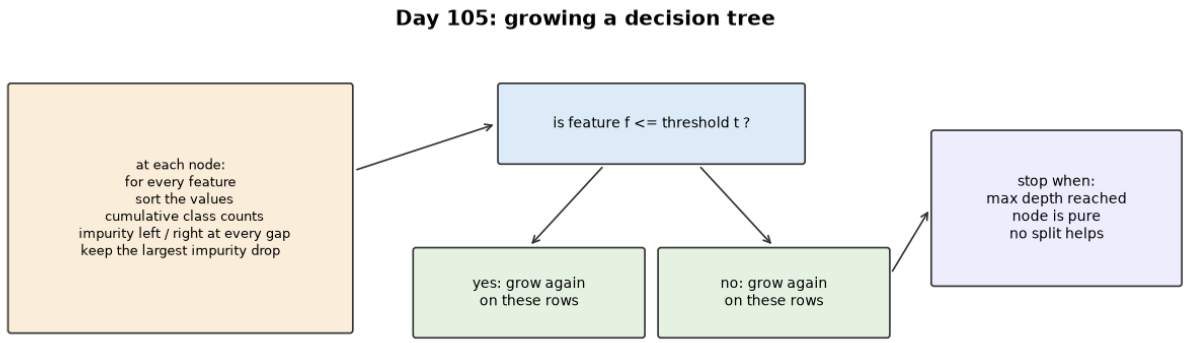
A **decision tree** classifies by asking one yes/no question about one feature at a time ('is the worst radius at most 16.8?'), sending each example left or right, until it reaches a **leaf** that predicts the majority class of the training examples that ended there.

Training chooses each question greedily: the one that makes the two groups **purest**, measured by an **impurity** — Gini or entropy — that is 0 for a group of one class and largest for an even mix.

REAL-WORLD SCENARIO

A triage nurse's flowchart: 'Fever above 38? Then: breathing difficulty? ...' — each question chosen to separate the urgent from the not-urgent as cleanly as possible.

SYSTEM DIAGRAM



Growing a tree: at each node, search every feature and threshold, split, and repeat on each side.

Week 16 and today's part for Day 111

Week 16 returns to classical machine learning with the two-track structure: Days 105-110 each add an exercise part, and Day 111 assembles them for a head-to-head of trees against neural networks on tabular data. `day105_exercise.py` is the impurity functions and the split search, checked on a table small enough to work out by hand. Its real output:

ACTUAL OUTPUT

```
Gini of the whole table (4 flu, 4 not): 0.500; entropy 1.000 bits
Gini    : best question 'feature 1 (temperature) <= 37.75', impurity removed 0.500 (temperature
    separates the classes perfectly, so all impurity goes)
entropy : best question 'feature 1 (temperature) <= 37.75', impurity removed 1.000 (temperature
    separates the classes perfectly, so all impurity goes)
a pure node: best_split -> None
identical feature values: best_split -> None
min_leaf = 5 on 8 rows (no split leaves 5 on both sides): best_split -> None
min_leaf = 3: still possible -> feature 1, threshold 37.75, impurity removed 0.500
```

Data: the Breast Cancer Wisconsin (diagnostic) set ships with scikit-learn, so nothing is downloaded; only its data loader is used, no scikit-learn models. The Titanic file is the repository's own Week 3 copy.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Impurity, and the root split

DEFINITION — start here, no prior knowledge assumed

For a group with class proportions p_k: **Gini** = $1 - \Sigma\, p_k^{\,2}$ (the chance that two random members have different classes) and **entropy** = $-\Sigma\, p_k \log_2 p_k$ (the average bits needed to name a member's class). A split's quality is the **impurity removed**: the parent's impurity minus the size-weighted impurity of the two children.

REAL-WORLD SCENARIO

Sorting laundry: a basket of only whites needs no more sorting; a 50/50 basket needs the most.

MATHEMATICS

$$Gini(S) = 1 - \Sigma_k\, p_k^{\,2}, \quad H(S) = -\Sigma_k\, p_k \log_2 p_k$$

$$gain = I(S) - (|L|/|S|)\, I(L) - (|R|/|S|)\, I(R), \quad L = \{x : x_f \le t\},\ R = the\ rest$$

IMPLEMENTATION

```
def gini(counts):
    n = counts.sum(-1, keepdims=True)
    p = counts / np.maximum(n, 1)
    return 1 - (p ** 2).sum(-1)
def entropy(counts):
    n = counts.sum(-1, keepdims=True)
    p = counts / np.maximum(n, 1)
    with np.errstate(divide="ignore", invalid="ignore"):
        return np.where(p > 0, -p * np.log2(p), 0.0).sum(-1)
```

ACTUAL OUTPUT

```
  a group 50% / 50%: Gini 0.500, entropy 1.000 bits
  a group 90% / 10%: Gini 0.180, entropy 0.469 bits
  a group 100% / 0%: Gini 0.000, entropy 0.000 bits
Breast cancer: 569 tumours, 30 features, 0.627 benign
  Gini     root split: worst radius <= 16.8 (impurity removed 0.3252); left [33, 346], right [179,
      11] (malignant, benign)
  entropy  root split: worst perimeter <= 106 (impurity removed 0.5620); left [17, 328], right
      [195, 29] (malignant, benign)
Actual output from this lesson's run.
```

What it means

Both measures are 0 for a pure group and largest at 50/50 (Gini 0.500, entropy 1.000 bit). On the full dataset they choose *different* first questions: Gini 'worst radius ≤ 16.8', entropy 'worst perimeter ≤ 106'. Radius and perimeter are nearly the same measurement (perimeter is about 2π times radius), so the two criteria agree on the idea and differ on which near-duplicate feature carries it.

WHY CHOOSE THIS?

Simple, fast to compute from counts, and the split is easy to explain.

WHY NOT THIS? (TRADEOFFS)

Greedy: the best first question does not guarantee the best tree.

2. The split search, checked

DEFINITION — start here, no prior knowledge assumed

The fast search sorts each feature once and gets the class counts left of every possible threshold with one cumulative sum, so all thresholds of a feature are scored at once. A brute-force version loops over every threshold and counts again. Both must find the same best gain.

REAL-WORLD SCENARIO

Checking a fast mental shortcut against slow long division on a few problems before trusting it.

IMPLEMENTATION

```
def best_split(X, y, n_classes, impurity, min_leaf=1):
    n, best = len(y), None
    parent = impurity(np.bincount(y, minlength=n_classes)[None])[0]
    for f in range(X.shape[1]):
        order = np.argsort(X[:, f], kind="stable"); xs, ys = X[order, f], y[order]
        left = np.cumsum(np.eye(n_classes, dtype=int)[ys], axis=0)[:-1]
        right = np.bincount(y, minlength=n_classes) - left
        nl = np.arange(1, n)
        ok = (xs[1:] != xs[:-1]) & (nl >= min_leaf) & (n - nl >= min_leaf)
        if not ok.any():
            continue
        child = (nl * impurity(left) + (n - nl) * impurity(right)) / n
        gain = np.where(ok, parent - child, -np.inf)
        i = int(np.argmax(gain))
        if best is None or gain[i] > best[2] + 1e-12:
            best = (f, (xs[i] + xs[i + 1]) / 2, float(gain[i]))
    return best if best and best[2] > 1e-12 else None
```

ACTUAL OUTPUT

```
30 random problems (25 rows, 3 features, 3 classes): the fast search finds the brute-force best
    gain in 30 of 30
Actual output from this lesson's run.
```

What it means

The fast search matches brute force in 30 of 30 random problems with repeated values and three classes. Thresholds are placed only between *different* neighbouring values, so a split never separates identical values.

WHY CHOOSE THIS?

Scoring all thresholds at once makes a full tree on 400 rows take a fraction of a second.

WHY NOT THIS? (TRADEOFFS)

Memory grows with rows times classes per feature; very large data needs histograms instead.

3. Depth: training and test accuracy

DEFINITION — start here, no prior knowledge assumed

The data is split once into 398 training and 171 test tumours, keeping the class balance (stratified). Trees are grown with Gini to maximum depths 1 to 10 and without limit.

REAL-WORLD SCENARIO

A student who memorises every past exam paper scores 100% on them and no better on a new one.

SPECIFICATION

Data	569 tumours, 30 features, 0 = malignant, 1 = benign.
Split	70/30, stratified, seed 0.
Tree	Gini, minimum leaf 1, thresholds between neighbouring values.

IMPLEMENTATION

```
def grow(X, y, n_classes, impurity, depth=0, max_depth=None, min_leaf=MIN_LEAF):
    counts = np.bincount(y, minlength=n_classes)
    node = {"counts": counts, "pred": int(np.argmax(counts))}
    if (max_depth is not None and depth >= max_depth) or counts.max() == len(y):
        return node
    s = best_split(X, y, n_classes, impurity, min_leaf)
    if s is None:
        return node
    f, t, g = s; m = X[:, f] <= t
    node.update(feature=f, threshold=t, gain=g,
                left=grow(X[m], y[m], n_classes, impurity, depth + 1, max_depth, min_leaf),
                right=grow(X[~m], y[~m], n_classes, impurity, depth + 1, max_depth, min_leaf))
    return node
```

ACTUAL OUTPUT

```
stratified split: 398 training, 171 test tumours (seed 0); Gini; minimum leaf 1
 max depth  leaves  depth  train acc  test acc
         1       2      1     0.9271    0.9064
         2       4      2     0.9573    0.9357
         3       8      3     0.9698    0.9240
         4      10      4     0.9874    0.9415
         5      12      5     0.9874    0.9415
         6      15      6     0.9925    0.9415
         7      18      7     0.9975    0.9415
         8      19      8     1.0000    0.9415
         9      19      8     1.0000    0.9415
        10      19      8     1.0000    0.9415
      None      19      8     1.0000    0.9415
Actual output from this lesson's run.
```

What it means

Training accuracy climbs to 1.0000 at depth 8, where the tree has 19 leaves and stops growing because every leaf is pure. Test accuracy reaches 0.9415 at depth 4 and never improves after that: the extra leaves fit individual training tumours. With 171 test tumours one more correct answer is worth 0.006, so depth 3's 0.9240 is about three tumours below depth 4.

WHY CHOOSE THIS?

Depth is one simple knob between too simple and too specific.

WHY NOT THIS? (TRADEOFFS)

Choosing it on the test set would leak; section 4 chooses it without looking.

4. Choosing the depth by 5-fold cross-validation

DEFINITION — start here, no prior knowledge assumed

k-fold cross-validation splits the training part into k folds, trains on k – 1 of them and measures on the one left out, k times, then averages. The depth with the best average is chosen, and only then is the test set used, once.

REAL-WORLD SCENARIO

Rehearsing a talk in front of five small groups instead of one: one group's reaction could be a fluke.

IMPLEMENTATION

```
def kfold_accuracy(X, y, impurity, max_depth, k=5, seed=0):
    folds = np.array_split(np.random.RandomState(seed).permutation(len(y)), k)
    accs = []
    for i in range(k):
        va = folds[i]; tr = np.concatenate([folds[j] for j in range(k) if j != i])
        t = grow(X[tr], y[tr], 2, impurity, max_depth=max_depth)
        accs.append(np.mean(predict(t, X[va]) == y[va]))
    return np.mean(accs), np.std(accs)
```

ACTUAL OUTPUT

```
  max depth    1: cross-validated accuracy 0.8945 (+- 0.0170 across folds)
  max depth    2: cross-validated accuracy 0.9220 (+- 0.0261 across folds)
  max depth    3: cross-validated accuracy 0.9170 (+- 0.0237 across folds)
  max depth    4: cross-validated accuracy 0.9170 (+- 0.0207 across folds)
  max depth    5: cross-validated accuracy 0.9196 (+- 0.0221 across folds)
  max depth    6: cross-validated accuracy 0.9171 (+- 0.0234 across folds)
  max depth    7: cross-validated accuracy 0.9146 (+- 0.0244 across folds)
  max depth    8: cross-validated accuracy 0.9146 (+- 0.0244 across folds)
  max depth    9: cross-validated accuracy 0.9146 (+- 0.0244 across folds)
  max depth   10: cross-validated accuracy 0.9146 (+- 0.0244 across folds)
  max depth None: cross-validated accuracy 0.9146 (+- 0.0244 across folds)
chosen: max depth 2; test accuracy 0.9357, 95% interval [0.8947, 0.9708] (Day 99's bootstrap)
Actual output from this lesson's run.
```

What it means

Cross-validation prefers depth 2 (0.9220), but depths 2 to 6 are within about half a point of each other while the folds differ by about 0.0261: the choice is close to a coin toss among shallow trees. The chosen tree scores 0.9357 on test, with a 95% interval of [0.8947, 0.9708]: the test set is too small to separate depth 2 from depth 4.

WHY CHOOSE THIS?

An honest choice that never touches the test set.

WHY NOT THIS? (TRADEOFFS)

k times the training cost, and still noisy on small data.

5. Gini against entropy

DEFINITION — start here, no prior knowledge assumed

The same trees grown with entropy instead of Gini, at the chosen depth and without limit.

REAL-WORLD SCENARIO

Two judges with slightly different scoring sheets usually, but not always, pick the same winner.

ACTUAL OUTPUT

```
  Gini     max depth    2:   4 leaves, test accuracy 0.9357
  Gini     max depth None:  19 leaves, test accuracy 0.9415
  entropy  max depth    2:   4 leaves, test accuracy 0.9474
  entropy  max depth None:  17 leaves, test accuracy 0.9649
  the two criteria agree on 0.9064 of test tumours at max depth 2
  the two criteria agree on 0.9415 of test tumours at max depth None
Actual output from this lesson's run.
```

What it means

Entropy scored higher here (0.9474 against 0.9357 at depth 2, 0.9649 against 0.9415 unlimited), but the two depth-2 trees agree on only 0.9064 of tumours, and every gap is inside the test set's noise. On this split entropy happened to do better; this one experiment cannot say it is better in general.

WHY CHOOSE THIS?

Gini is a little cheaper (no logarithm) and usually gives similar trees.

WHY NOT THIS? (TRADEOFFS)

Neither is reliably better; the choice rarely matters as much as depth.

6. Instability: the same recipe on 30 resamples

DEFINITION — start here, no prior knowledge assumed

A **bootstrap resample** draws 398 training tumours *with replacement* from the 398 (some twice, some not at all). Thirty resamples, thirty depth-2 trees, all scored on the same test set — and a majority vote of the thirty.

REAL-WORLD SCENARIO

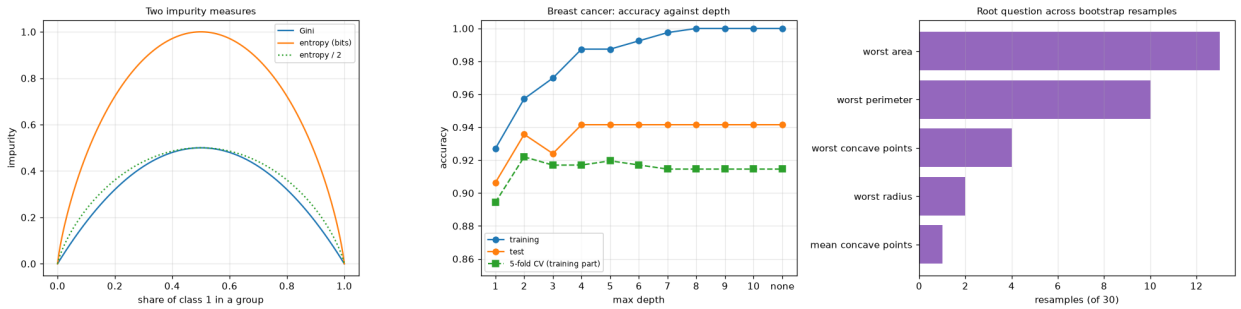
Thirty doctors, each trained on a slightly different set of past patients, will not all ask the same first question.

IMPLEMENTATION

```
for b in range(30):
    s = rng.randint(0, len(tr), len(tr)); idx = tr[s]
    t = grow(X[idx], y[idx], 2, gini, max_depth=best)
    roots[names[t["feature"]]] += 1; p = predict(t, X[te]); P.append(p); accs.append(np.mean(p ==
        y[te]))
P = np.array(P); vote = (P.mean(0) > 0.5).astype(int)
```

ACTUAL OUTPUT

```
root question chosen across resamples: worst area 13, worst perimeter 10, worst concave points 4,
    worst radius 2, mean concave points 1
test accuracy of single trees: mean 0.9376, min 0.9006, max 0.9649
test tumours on which the 30 trees disagree at all: 0.2515
a majority vote of the 30 trees (bagging, Day 107's subject): test accuracy 0.9649
Actual output from this lesson's run.
```



Left: Gini and entropy against the class mix. Middle: accuracy against maximum depth. Right: the first question chosen on each of 30 bootstrap resamples.

What it means

Small changes in the data change the tree: the first question uses five different features across 30 resamples, single trees range from 0.9006 to 0.9649, and the trees disagree somewhere on 0.2515 of test tumours. Their majority vote scores 0.9649, above the average single tree (0.9376). That is **bagging**, and averaging away this instability is the whole idea of Day 107's random forest.

WHY CHOOSE THIS?

Shows the variance of a single tree directly.

WHY NOT THIS? (TRADEOFFS)

One test split; the vote's advantage needs Day 107's fuller test.

7. A readable tree: Titanic

DEFINITION — start here, no prior knowledge assumed

Titanic passengers (class, sex, age with missing values filled by the median, relatives aboard, fare), 70/30 split, a depth-3 tree printed as rules. Each leaf shows [died, survived] among training passengers.

REAL-WORLD SCENARIO

A flowchart a museum could put on the wall next to the lifeboats exhibit.

ACTUAL OUTPUT

```
891 passengers (623 training, 268 test), 0.384 survived; Gini, max depth 3; counts are [died,
    survived] among training passengers
  if is_female <= 0.5:
      if Age <= 5.5:
          if SibSp <= 2.5:
              -> survived  [0, 10]
          else:
              -> died  [3, 0]
      else:
          if Pclass <= 1.5:
              -> died  [53, 30]
          else:
              -> died  [274, 35]
```

```
  else:
      if Pclass <= 2.5:
          if Age <= 3:
              -> died  [1, 0]
          else:
              -> survived  [6, 110]
      else:
          if Fare <= 23.4:
              -> survived  [29, 51]
          else:
              -> died  [18, 3]
test accuracy: tree 0.8022; 'women survive, men do not' 0.7724
Actual output from this lesson's run.
```

What it means

The tree reads like history: sex first, then age for men (young boys survived) and class for women. It beats the one-rule baseline 'women survive' (0.8022 against 0.7724). It also shows the danger of pure leaves: one leaf, [1, 0], is a single passenger — a rule made from one example. Limiting leaf size or pruning (Day 106) removes such leaves.

WHY CHOOSE THIS?

Rules a person can check and argue with.

WHY NOT THIS? (TRADEOFFS)

Filling missing ages with the median is a simplification; the tree trusts it.

Interview preparation — technical questions

Q: How is the best split found efficiently?

A: Sort each feature once; cumulative class counts give every threshold's left and right counts at once.

Q: Where are thresholds placed?

A: Halfway between neighbouring different values, so equal values are never separated.

Q: When does a tree stop growing?

A: Maximum depth, a pure node, a minimum leaf size, or no split that reduces impurity.

Q: How do you choose the depth?

A: Cross-validation on the training part; here depth 2 at 0.9220, the test set used once.

Q: Gini or entropy?

A: Usually similar; here the depth-2 trees agreed on 0.9064 of tumours.

Interview preparation — theoretical questions

Q: Why does a fully grown tree overfit?

A: It keeps splitting until every leaf is pure, fitting single training examples: 1.0000 train, 0.9415 test.

Q: Why are trees unstable?

A: A small data change can change the first split, and everything below it changes too.

Q: Why does averaging trees help?

A: Their errors differ; the vote cancels some (0.9649 vs 0.9376 on average here).

Q: Why might trees beat neural networks on tabular data?

A: Axis-aligned questions suit features with different units and sharp thresholds; Day 111 tests it.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A decision tree is a greedy search for the purest yes/no questions, and it is a few dozen lines of NumPy. On real data it reaches about 0.94 test accuracy, fits its training data perfectly if allowed, and changes shape with small changes in the data. Depth control fights the first problem; averaging many trees fights the second.

- Root split: Gini 'worst radius <= 16.8', entropy 'worst perimeter <= 106'.
- Unlimited depth: 19 leaves, training 1.0000, test 0.9415; CV picks depth 2, test 0.9357 [0.8947, 0.9708].
- 30 bootstrap trees: test 0.9006 to 0.9649; their vote 0.9649.
- Titanic depth-3 tree 0.8022 against 'women survive' 0.7724.

Suggested watch order, if starting from zero

- 1. Decision and Classification Trees, Clearly Explained!!! (StatQuest) — how a classification tree is built, step by step.
- 2. Gini & Entropy versus misclassification error (Sebastian Raschka) — why Gini and entropy are used instead of the error rate.

Day 106 preview

- **Pruning.** Grow the full tree, then cut it back (cost-complexity pruning) and watch over- and underfitting on real data.